

MRを作成したらAIがレビューと単体テスト作成を自動で支援してくれるシステム

企画書 要約

レビューアの指摘負担を減らし、レビューイの修正コストも下げる。AIが双方の橋渡しをする。

1. 概要

GitLab・Jenkins・CLIEージェントを連携し、
MR作成時のコードレビュー・修正・ユニットテスト生成・検証を**AIが支援するシステム**。

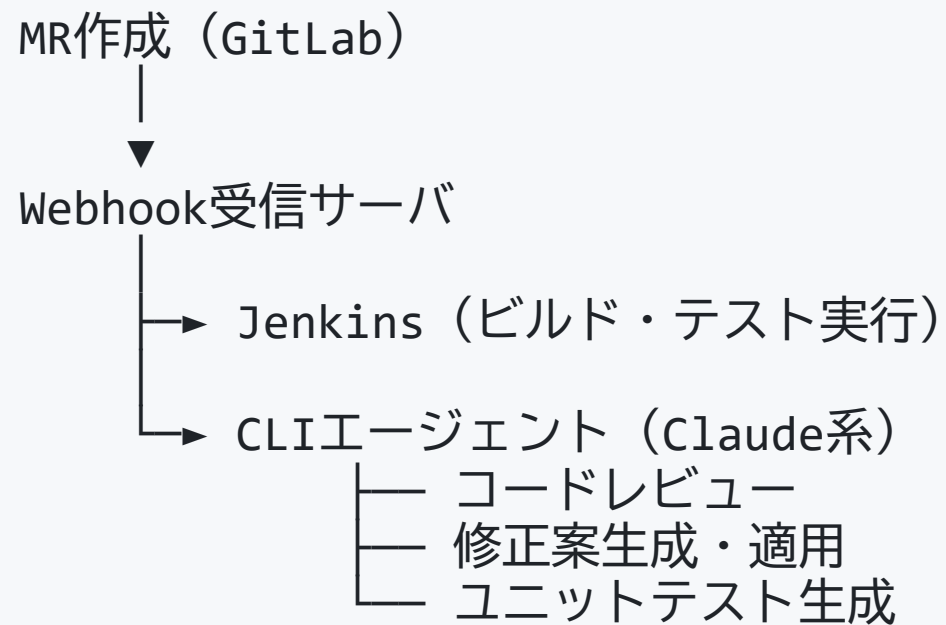
解決する課題：

レビュアーの指摘作業コスト / レビュー어의修正・対応コスト
→ AIが双方の橋渡しをすることで**品質向上サイクルを効率化**

2. コンセプト

軸	内容
差分理解型AI	コードの変更点を深く理解したレビュー
人間とAIの協調	完全自動ではなく 半自動 で安全性を確保
既存CI/CDへの統合	フローを壊さず自然に組み込む

3. システム構成

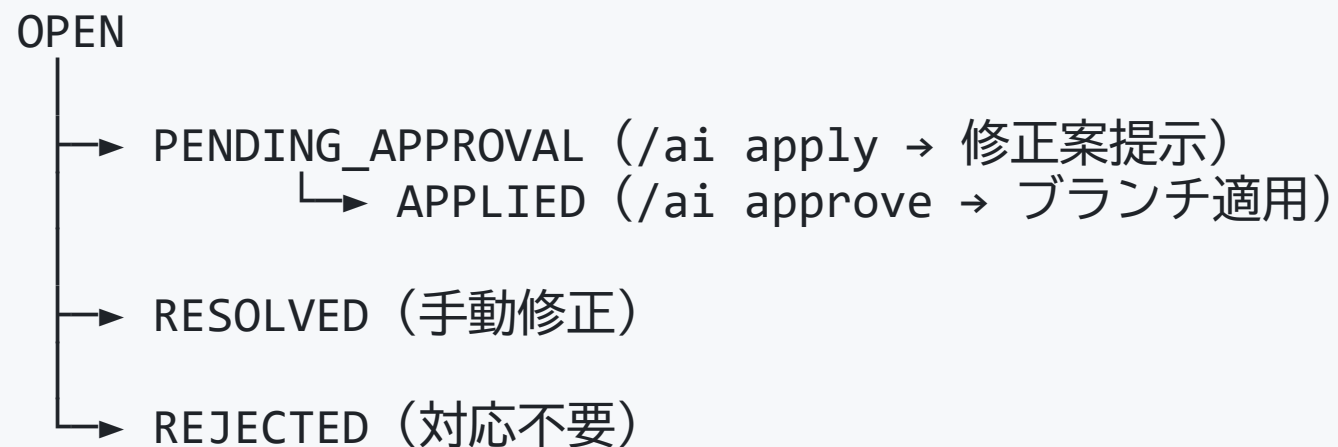


最小構成： 1台のサーバで全コンポーネントを実行可能

4. 処理フロー（全体）

- ① MR作成
- ② Jenkinsビルド（失敗時はレビュー不実施）
- ③ AIレビュー実行
- ④ 人間レビュー
- ⑤ 修正フェーズ（AI適用 or 手動）
- ⑥ 全指摘Close（対応完了の合図）
- ⑦ 再レビュー実行
- ⑧ ユニットテスト生成
- ⑨ Jenkinsでテスト実行
- ⑩ 結果をMRへフィードバック

5. 指摘管理モデル



全指摘のGitLab DiscussionがClose (Resolve) された時点で
再レビュー → ユニットテスト生成を自動起動

6. UI設計（GitLabコメントベース）

追加UIなし。GitLabのコメント機能のみで操作完結。

```
# 開発者・レビューア操作
[GitLabコメントのClose]    # 指摘への対応完了の合図
                           # 全Close後、再レビュー→ユニットテスト生成が自動起動

# 開発者操作
/ai apply R1                # 修正案を生成・提示
/ai approve R1              # 修正案を承認・適用
/ai reject R2               # 指摘を対応不要と判断
/ai test                    # ユニットテスト生成を手動トリガー
```

7. ユニットテスト生成戦略

生成タイミング

全指摘Close後の再レビューで問題なしと判定された時点

生成内容

テスト種別	目的
バグ再現テスト	修正前に失敗するケース、再発防止
正常・境界テスト	分岐網羅、エッジケース検証

既存テストとの連携

カバレッジ取得 → 差分未カバー箇所を抽出 → AIが不足分のみ補完

8. 自律性設計

自動トリガー

- MR作成 → ビルド・AIレビュー実行
- 全指摘Close → 再レビュー実行 → ユニットテスト生成
- `/ai approve` → ブランチ適用

人間が関与するゲート

- 人間レビュー：AI指摘の確認
- 修正承認：`/ai approve` による明示的承認
- 最終マージ：GitLab の通常Approveフロー

9. 期待効果

効果	内容
レビュー工数削減	AIによる一次レビューの自動化
品質の均一化	レビュー基準の標準化
バグ再発防止	修正に対応したテストを自動生成
属人性の低減	レビューアー依存の排除
AI活用度の統一	共通システムを通じてAIの活用度合いを正規化。使用者ごとのレビュー精度のバラツキを解消し、チーム全体で品質・粒度を揃える

まとめ

レビューアーは判断業務に集中できる

レビュイーはAIと共に修正を進められる

→ 双方の負担を下げながら、品質向上サイクルを回し続ける

- GitLab ネイティブな操作感で導入障壁を低減
- 人間の判断を残しつつ、繰り返し作業をAIが担う
- 1台構成から始められるスモールスタート設計